Faculty of Computing

Year 2 Semester 2 (2026)

IT2130 - Operating Systems and System Administration — Lab Sheet 4

Objectives: In the end of this practical, the students will be able to understand process management system calls and implement process management concepts using C language.

A process is a program in execution and is considered an active entity in a computer system. Processes require various resources for execution, such as CPU time, main memory, secondary storage, input/output devices, and files. The management of processes is a fundamental responsibility of the operating system.

The operating system provides process management services to users through system calls. These system calls are implemented within the kernel and require a transition from user mode to kernel mode for execution.

Table 1: Details about Process Management System

System call	Function
fork ( )	Create processes in the UNIX system
getpid ( )	Get the process ID of the process
getppid ( )	Get the parent process ID of the process
exit ( )	Exit the process execution
wait ( )	Parent process waits until the child processes are terminating
execl ( )	This loads and executes a new process, replacing the currently running process, while keeping the same process ID

Printing process IDs of Processes

Activity 1: Consider the below program and answer the questions.

```
#include<stdio.h>
#include<unistd.h>
int main()
{
     printf ("Process ID is: %d\n", getpid() );
     return 0;
}
```

a. What is the output you can see in the terminal?

b. What is the system call used in the above program?

c. Briefly describe the purpose of the system call you mentioned in a)?

d. Remove the include statement, "#include¡unistd.h¿" from the program and check the output. What can you see in the output? Describe the reason for your observation.

Get the manual page of getpid. This fetches the process identifier/ process ID of the current process.

To get the manual page type: *man getpid*

Activity 2:

Modify the program in activity 1 and print the parent process ID of the current process. Include the modified program and its output into the submission file.

Note: Invoke the getppid ( ) system call to get the parent process id.

Sample output

```
admin@Ubuntu:~$ ./lab4
Process ID is: 2769
Process ID of parent process is: 2692
admin@Ubuntu:~$
```

Figure 1: Representation of process ID and parent process ID of the process

As figure 1 illustrates, the process name is lab4. The process ID is 2769 and it's parent process ID is 2692.

View the Process Hierarchy

The UNIX operating system organizes its processes in a hierarchical structure. This hierarchy has a single root process, and all other processes are organized in a parent-child relationship.

Activity 3:

> **The `sleep( )` system call is used to suspend the execution of a process for a specified amount of time. During this period, the process is put into a waiting state. After the given number of seconds expires, the process resumes execution from where it was paused.**
>
> **Read the manual page of `sleep()`. The `sleep(20)` refers to the process suspends for 20 seconds.**

1. Modify the program in activity 2, and add the system call, sleep( ) after printing the parent process ID. Let the current process to sleep 30 seconds.
2. Then open a new terminal and type the below UNIX command.
   pstree -p

Now you can see a hierarchy of processes printed on the terminal. Open the manual page of pstree and understand the behavior of it.

Sample output

```
admin@Ubuntu:~$ ./lab4
Process ID is: 2769
Process ID of parent process is: 2692
admin@Ubuntu:~$
```

```
├─gnome-terminal-(2685)─┬─bash(2692)──lab4(2769)
                        ├─bash(2770)──pstree(2783)
```

Figure 2: Representation of the process ID and parent process ID in the process tree

Figure 2 illustrates, the ID of the process is 2769 and the ID of the parent process as 2692. In the process tree the processes are listed with the IDs as a tree. Accordingly, the name of the parent process is bash. It is the shell you are working in your UNIX system.

To see the process tree during the lab4 process is sleeping, a separate terminal/ shell is opened.

Exit a Process

The exit( ) system call ends the currently running process. It returns an exit status value to the parent process, which can be used to determine whether the process terminated successfully or encountered an error.

If exit status value is 0 (`status = 0`), it represents the normal termination of the process and if the status is a non-zero value (status $\neq 0$), $itisanabnormaltermination.$

Activity 4:

Type the program given below and understand the behavior of exit (0). Explain the reason for not printing the message, "This line will not be executed".

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Program is running...\n");

    exit (0);   // Terminates the program normally

    printf("This line will not be executed.\n");
    return 0;
}
```

Process Creation

The *fork()* system call is used in UNIX to create a new process. When *fork()* invokes, it creates a child process that is an exact copy of the parent process. After the call, both parent and child continue the execution from the same point, but *fork()* returns 0 to the child process, the child's process ID to the parent, and -1 if the creation fails.

Activity 5:

Type the below program and execute. Observe the output and explain the reason for printing "Hello world" twice.

```
#include <stdio.h>
int main()
{
     printf("I am parent\n");
     fork();
     printf("Hello world...!\n");
     return 0;
}
```

```
I'm parent...
Hello world
Hello world
```

Figure 3: Output of Program in Activity 5

Activity 6:

Modify the program in activity 5, to do the below.

1. Invoke the fork ( ) system call to create a child process.
2. Print the return value of the fork ( ).
3. Separate the parent and the child processes and print their process IDs as shown in figure below.

```
I'm the child process. My PID is: 4742
I'm the parent process. My PID is: 4741
```

Figure 4: Output for Activity 6

4. Add the sleep ( ) with an appropriate time in seconds and check the process hierarchy.

Replace the current process image with a new program

The *execl()* system call loads and executes a new program within the same process without creating a new process. Instead of starting a separate process, it replaces the existing process's code, data, and stack with those of the new program.

If *execl()* executes successfully, the original program does not continue its execution. However, if the call fails, it returns -1 and the original program continues to run.

Activity 7:

Type the below program and execute. Observe the output and explain the behavior of execl().

```
#include <stdio.h>
#include<unistd.h>
int main()
{
  printf("here comes the date. \n");
  execl("/bin/date", "date", null);    //null means end of
                                       arguments
  printf("that was the date. \n");
  return 0;
}
```

Activity 8:

Following program demonstrates the sue of system() library function. It is not a system call but, internally uses system calls to execute commands

Type the program given below and execute. Observe the behavior of system().

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   printf("here comes the date. \n");
   system("date");
   printf("that was the date");
   return 0;
}
```

Modify the above program by adding fork () prior to system () function. Observe the output and explain the output.